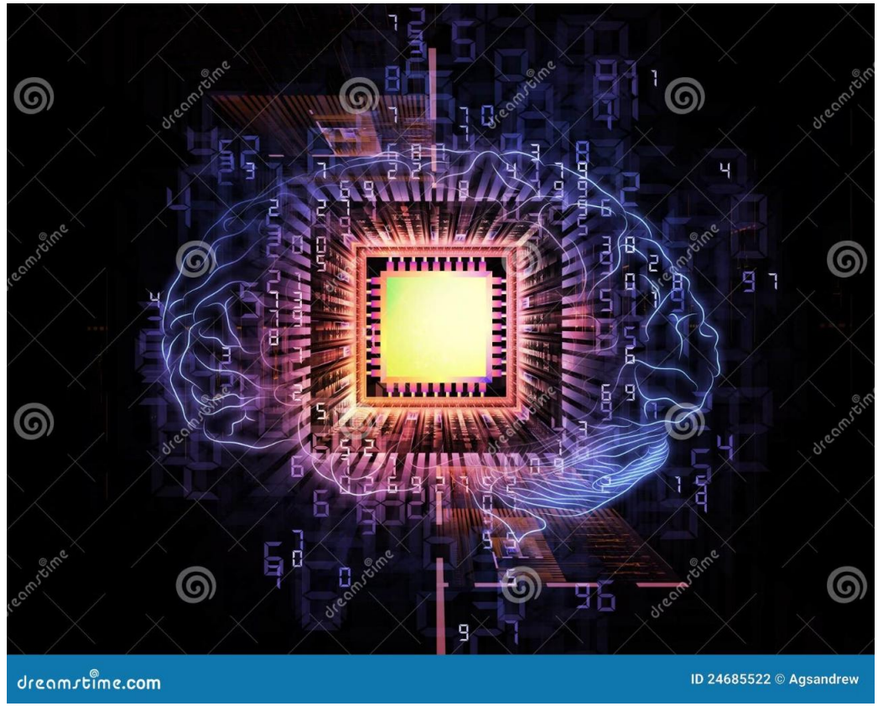
Sayı Sistemleri ve Dijital Mantık

Bu sunumun ana hedefi, öğrencilerin yazdıkları kodun ve kullandıkları verinin—sayılar, karakterler, görüntüler—bilgisayarın "beyninde" nasıl 1'lere ve 0'lara dönüştüğünü kavramalarını sağlamaktır. Dijital mantığın, modern hesaplamaların ve aritmetik işlemlerin temelini nasıl oluşturduğunu keşfedeceğiz.

Neden Sayı Sistemleri?

Biz insanlar günlük yaşamda 10'luk (ondalık) sistemde düşünür ve sayarız. Bu bizim için doğal ve sezgiseldir. Ancak bilgisayarlar, milyonlarca transistörden oluşur ve bu transistörlerin her biri yalnızca iki durumda bulunabilir: **"elektrik var"** (1) veya **"elektrik yok"** (0).

Bu fiziksel gerçeklik, bilgisayarların tüm bilgiyi ikili (binary) sistemde temsil etmesini zorunlu kılar. Onluk düşünce yapımızı ikili gerçekliğe çevirebilmek, hem yazılımın temellerini (veri tipleri, bellek yönetimi) hem de donanımın işleyişini (aritmetik devreler, mantık kapıları) anlamak için kritik öneme sahiptir.



Konumsal Sayı Sistemleri: Genel İlke

Bir sayının değeri, o sayıyı oluşturan rakamların konumuna bağlıdır. Bu, tüm konumsal sayı sistemlerinin temel prensibidir.

Matematiksel İfade

Taban **b** için bir sayı şu şekilde ifade edilir:

$$\sum_{i=0}^{n} d_i \times b^i$$

Burada **di** rakamları, **b** tabanı ve **i** konumu temsil eder.

Onluk Sistem Örneği

347 sayısını inceleyelim:

$$347_{10} = 3 \times 10^2 + 4 \times 10^1 + 7 \times 10^0$$

$$= 300 + 40 + 7 = 347$$

İkili Sistem Örneği

Aynı mantık ikili sistemde:

$$101_2 = 1 \times 2^2 + 0 \times 2^1 + 1 \times 2^0$$

$$= 4 + 0 + 1 = 5_{10}$$

İkili Sistem: Onluktan İkiliye Dönüşüm

Dönüşüm Algoritması

Bir onluk sayıyı ikiliye çevirmek için ardışık bölme yöntemi kullanırız:

1. Onluk sayıyı sürekli 2'ye böl
2. Her adımda **kalanı** kaydet
3. Kalanları **tersten** yaz

Bu yöntem, sayının ikili temsilini bit bit oluşturur. Her bölme işlemi, bir sonraki düşük anlamlı biti verir.

Örnek: 45'i İkiliye Çevirme

Bölme	Sonuç	Kalan
45 ÷ 2	22	1
22 ÷ 2	11	0
11 ÷ 2	5	1
5 ÷ 2	2	1
2 ÷ 2	1	0
1 ÷ 2	0	1

Sonuç (tersten): $101101_2 = 45_{10}$



Onaltılık Sistem ve Pratik Kullanımı

İkili sistem sayıları çok uzun hale getirir ve insanlar için okunması zordur. Örneğin, 16-bit'lik 1011010100101011 sayısını hatırlamak neredeyse imkansızdır.

Onaltılık Nedir?

Onaltılık (hexadecimal) sistem, 16 farklı sembol kullanır: **0-9** ve **A-F** (A=10, B=11, ..., F=15). Bu sistem, ikili sistemin "okunabilir kısaltması" gibi çalışır.

4-Bit Grupları

$2^4 = 16$ olduğu için, her 4 bitlik ikili grup tam olarak 1 hex rakamına karşılık gelir. Bu, dönüşümü son derece basit hale getirir.

Örnek: 1101 0110 1010 1111 → D 6 A F → $D6AF_{16}$

Gerçek Dünya Kullanımı

Hex format, bilgisayar biliminde yaygın kullanılır:

- **Bellek adresleri:** 0x7FFF8A20
- **Renk kodları:** #FF0000 (Kırmızı)
- **MAC adresleri:** 00:1A:2B:3C:4D:5E

İşaretli Tamsayılar: İşaret-Büyüklük Yöntemi

İlk Yaklaşım

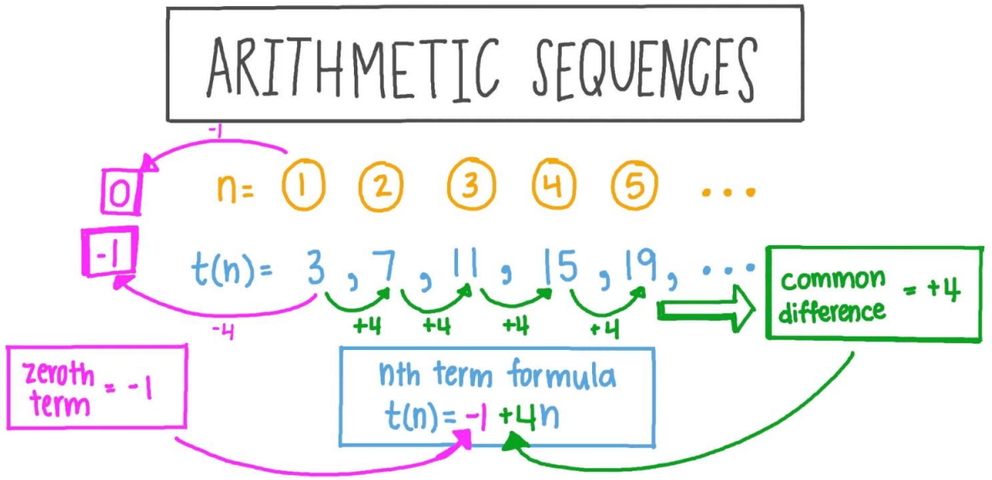
Pozitif sayıları temsil etmeyi öğrendik. Peki negatif sayılar nasıl saklanır?

En sezgisel yaklaşım **işaret-büyüklük (sign-magnitude)** yöntemidir:

- En soldaki bit **işaret biti** olur
- 0 = pozitif sayı
- 1 = negatif sayı
- Geri kalan bitler büyüklüğü gösterir

Örnek (4-bit):+5 → 0101-5 → 1101

Kritik Sorunlar



Bu yöntemin ciddi dezavantajları vardır:

1. İki farklı sıfır: +0 (0000) ve -0 (1000) ayrı temsil edilir
2. **Karmaşık donanım:** Toplama ve çıkarma için farklı devreler gerekir
3. **Verimsizlik:** Aritmetik işlemler yavaş ve hatalara açıktır

Bu nedenle modern bilgisayarlar bu yöntemi **kullanmaz**.

2'nin Tümleyeni: Modern Standart

Modern bilgisayarların tamamı negatif tamsayılar için 2'nin tümleyeni (two's complement) yöntemini kullanır. Bu yöntem, işaret-büyüklük yönteminin tüm sorunlarını çözer.

Tek Sıfır

Sadece bir sıfır temsili vardır: 0000. Bu, karşılaştırma işlemlerini basitleştirir ve mantık hatalarını önler.

Aynı Donanım

Toplama ve çıkarma için **aynı basit devre** kullanılır. Bu, işlemci tasarımını hem daha basit hem de daha hızlı yapar.

Kolay Dönüşüm

Bir sayının negatifini almak için iki adım:**1)** Tüm bitleri ters çevir (NOT)**2)** Sonuca 1 ekle

---

Örnek: +5'in Negatifini Bulma (4-bit)

Adım 1: Başlangıç

+5 = 0101

Adım 2: Ters Çevir

NOT(0101) = 1010

Adım 3: 1 Ekle

1010 + 1 = 1011Bu -5'tir!

2'nin Tümleyeninin Gücü: Çıkarma = Toplama

A - B = A + (-B)

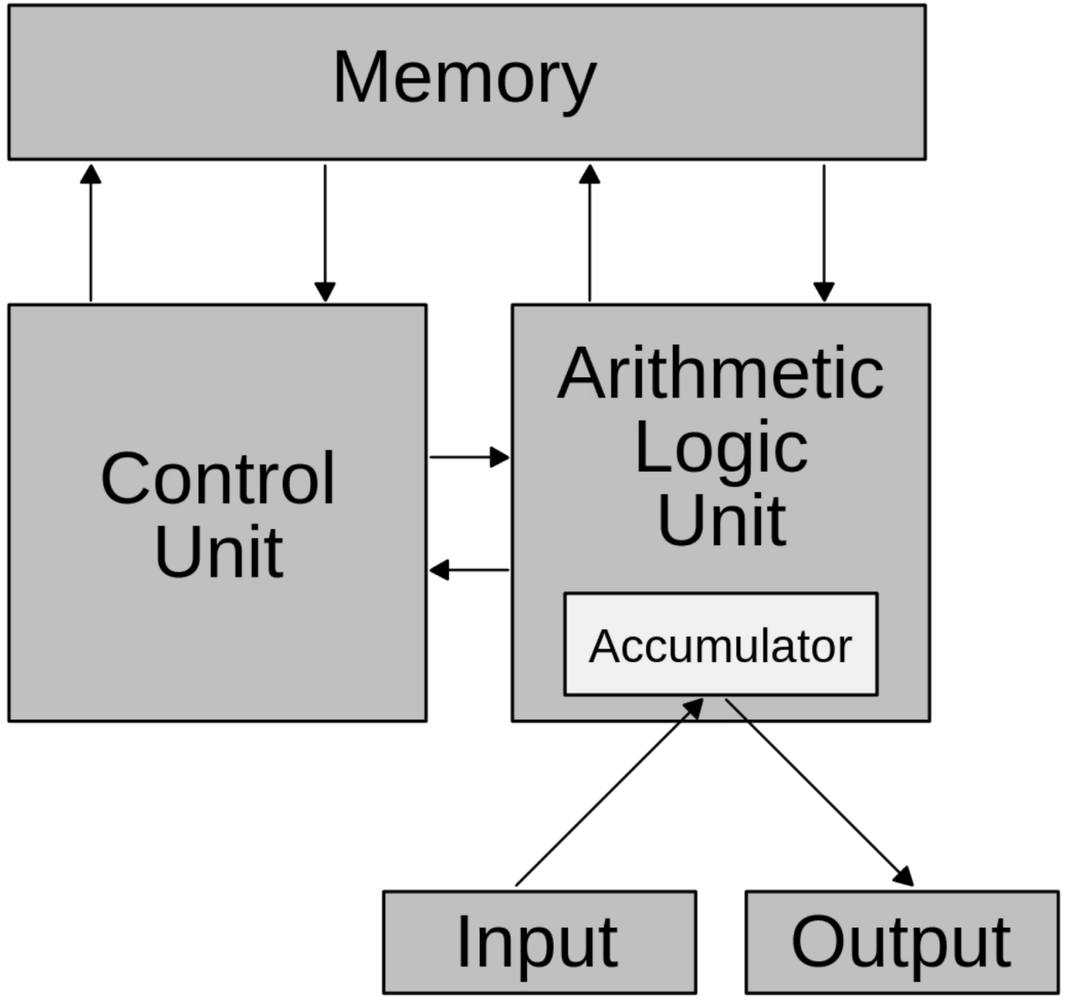
2'nin tümleyeni yönteminin en büyük avantajı, çıkarma işleminin donanımda **toplama işlemine dönüştürülmesidir**. Bu, işlemci tasarımını radikal şekilde basitleştirir.

Örnek: 7 - 5

Bu işlemi 7 + (-5) olarak gerçekleştireceğiz:

- 7 = 0111
- -5 = 1011 (önceki slaytta bulmuştuk)

Toplama:0111 (+7)+ 1011 (-5)------1 00104-bit sistemdeyiz, taşan 1'i atarız. Sonuç: 0010 = 2



Donanım Tasarımı İçin Sonuç



Taşma (Overflow) ve Sınır Durumları

Her veri tipinin bir kapasitesi vardır. 8-bit'lik işaretli bir tamsayı (2'nin tümleyeni ile) -128 ile +127 arasındaki değerleri temsil edebilir. Peki bu sınırları aşarsak ne olur?

Taşma Örneği

100 + 50 = 150 işlemini yapalım. Ancak 150, maksimum değer olan +127'yi aşar. Bilgisayar bu durumda **yanlış bir sonuç** üretir (genellikle negatif bir sayı).

Taşmayı Tespit Etme

2'nin tümleyeninde taşma, **işaret bitlerini** kontrol ederek anlaşılır:

- İki **pozitif** sayı toplandığında sonuç **negatif** (işaret biti 1) çıkarsa → Taşma
- İki **negatif** sayı toplandığında sonuç **pozitif** (işaret biti 0) çıkarsa → Taşma

Yazılımda Önlem

Programcılar, kritik hesaplamalarda:

- Daha büyük veri tipleri kullanmalı (16-bit, 32-bit)
- İşlem öncesi sınır kontrolleri yapmalı
- Overflow flag'lerini kontrol etmeli

Kayan Noktalı Sayılar: IEEE 754 Standardı

Tamsayıları (int) inceledik. Peki ondalıklı sayılar (float, double) bilgisayarda nasıl saklanır? Bu, çok daha karmaşık bir konudur.

Bilimsel Gösterim Benzeri

Kayan noktalı sayılar, **bilimsel gösterime** benzer bir formatta saklanır:

$$\text{Sayı} = (-1)^{\text{İşaret}} \times \text{Kesir} \times 2^{\text{Üs}}$$

Örneğin: *1.01101 |times 2^5*

Bu format, hem çok küçük (0.0000001) hem de çok büyük (1000000000) sayıları temsil edebilir.

32-bit Float Yapısı

Bileşen	Bit Sayısı	Aralık
İşaret	1 bit	± (0/1)
Üs (Exponent)	8 bit	-126 ile +127
Kesir (Mantissa)	23 bit	1.xxx...

Toplam: 32 bit = 4 byte

Kayan Noktalı Sayıların Tehlikesi: Hassasiyet Hatası

0.1 + 0.2 ≠ 0.3

IEEE-754 formatının en önemli ve şaşırtıcı sonucu: Bilgisayarlar ondalıklı sayıları **tam olarak temsil edemeyebilir**. Bu, yazılım geliştirmede kritik bir gerçektir.

Temel Sorun

0.1 sayısı, ikili sistemde **sonsuz tekrar eden** bir ondalık sayıya karşılık gelir (tıpkı 1/3'ün onluk sistemde 0.333... olması gibi).

Sonuç: 0.1 tam olarak temsil edilemez, en yakın yaklaşım saklanır.

Gerçek Sonuç

```
Python'da:>>> 0.1 + 0.20.30000000000000004>>>
0.1 + 0.2 == 0.3False
```

Bu bir hata değil, IEEE-754'ün doğal bir sonucudur!

Yazılımda Önlemler

Asla float == float karşılaştırması yapmayın!

Bunun yerine:

- Epsilon karşılaştırması: `|a - b| < 0.0001`
- Finansal hesaplarda: Decimal veri tipi
- Bilimsel hesaplarda: Hata payı analizi

Boolean Cebri: Mantığın Temeli

Boolean cebri, 1 (Doğru) ve 0 (Yanlış) değerleri üzerine kurulu bir matematiksel sistemdir. Tüm dijital devrelerin ve programlama mantığının temelidir.

AND (VE) - Mantıksal Çarpma

Sembol: A · B veya A ∧ B

Kural: İkisi de 1 ise sonuç 1, diğer tüm durumlarda 0.

A	B	A·B	Açıklama
0	0	0	-
0	1	0	-
1	0	0	-
1	1	1	Her ikisi de doğru

Fiziksel benzetme: Seri bağlı anahtarlar - her ikisi de kapalı olmalı.

OR (VEYA) - Mantıksal Toplama

Sembol: A + B veya A ∨ B

Kural: Herhangi biri 1 ise sonuç 1, ikisi de 0 ise 0.

A	B	A+B	Açıklama
0	0	0	Her ikisi de yanlış
0	1	1	En az biri doğru
1	0	1	En az biri doğru
1	1	1	En az biri doğru

Fiziksel benzetme: Paralel bağlı anahtarlar - herhangi biri kapalı olabilir.

NOT (DEĞİL) - Mantıksal Tersleme

Sembol: A' veya ¬A veya ~A

Kural: 1 ise 0 yap, 0 ise 1 yap.

A	A'
0	1
1	0

Fiziksel benzetme: Tersleyici anahtar - açık kapalı, kapalı açık.

De Morgan Yasaları

De Morgan yasaları, Boolean cebirinin en önemli sadeleştirme kurallarından biridir. Bu yasalar, karmaşık mantıksal ifadeleri daha basit formlara dönüştürmemizi sağlar.

Birinci Yasa

$$\overline{A \quad B} = \overline{A} + \overline{B}$$

"AND'in tersi, terlerin OR'una eşittir"

Örnek: "Hem yağmur yağmıyor hem rüzgar esmiyor" = "Ya yağmur yağmıyor ya rüzgar esmiyor"

İkinci Yasa

$$\overline{A + B} = \overline{A} \quad \overline{B}$$

"OR'un tersi, terlerin AND'ine eşittir"

Örnek: "Ne yağmur yağıyor ne kar yağıyor" = "Yağmur yağmıyor VE kar yağmıyor"

Doğruluk Tablosu ile Kanıt (Birinci Yasa)

A	B	A·B	¬(A·B)	¬A	¬B	¬A+¬B
0	0	0	1	1	1	1
0	1	0	1	1	0	1
1	0	0	1	0	1	1
1	1	1	0	0	0	0

Son iki sütun özdeştir - yasa kanıtlanmıştır! ✓

Kritik Önemi: De Morgan yasaları, karmaşık devreleri sadeleştirmemizi ve bir ifadeyi sadece AND/NOT veya sadece OR/NOT kullanarak yazmamızı sağlar. Bu, donanım tasarımında maliyet ve alan tasarrufu demektir.

Mantıksal Kapılar: Teoriden Donanıma

Boolean cebirindeki işlemleri (AND, OR, NOT) fiziksel olarak gerçekleştiren devre elemanlarıdır. Transistörlerle inşa edilirler ve tüm dijital devrelerin temel yapı taşlarıdır.

AND Kapısı

Her iki giriş de 1 ise çıkış 1

Sembol: D şeklinde kapı

OR Kapısı

En az bir giriş 1 ise çıkış 1

Sembol: Sivri uçlu kapı

NOT Kapısı

Girişi tersine çevirir

Sembol: Üçgen + daire

XOR Kapısı

Girişler farklıysa 1 (Özel VEYA)

Toplayıcılarda kritik rol oynar

NAND Kapısı

AND'in tersi (NOT-AND)

Evrensel kapı: Her devre NAND'larla yapılabilir

NOR Kapısı

OR'un tersi (NOT-OR)

Evrensel kapı: Her devre NOR'larla yapılabilir

İlginç Bilgi: NAND ve NOR kapıları **işlevsel tamlık** özelliğine sahiptir. Yani, *tüm dijital devreler* sadece NAND kapıları veya sadece NOR kapıları kullanılarak inşa edilebilir. Bu, üretim süreçlerinde büyük avantaj sağlar.

Kombinasyonel Devreler

Kombinasyonel devreler, çıkışı sadece o anki girişlere bağlı olan devrelerdir. Bu devrelerin "hafızası" yoktur - geçmiş girişleri "hatırlamazlar".

Üç Eşdeğer Temsil

01

Doğruluk Tablosu

Tüm olası giriş kombinasyonları ve karşılık gelen çıkışlar

02

Boolean İfadesi

Mantıksal operatörlerle yazılmış matematiksel formül

03

Kapı Devresi

Mantıksal kapılarla çizilmiş fiziksel şema

Bu üç temsil birbirine **tamamen eşdeğerdir** ve aralarında dönüşüm yapılabilir. Devre tasarımı genellikle doğruluk tablosundan başlar.

Örnek: Çoğunluk Devresi

3 girişli bir devre tasarlayalım. Çıkış, girişlerin **çoğunluğu 1 ise** 1 olsun.

A	B	C	Out
0	0	0	0
0	0	1	0
0	1	0	0
0	1	1	1
1	0	0	0
1	0	1	1
1	1	0	1
1	1	1	1

Boolean ifade:

$$Out = (A \quad B) + (B \quad C) + (A \quad C)$$

Yarım ve Tam Toplayıcı

İkili sayıları nasıl toplarız? İşte aritmetik devrelerin en temel yapı taşları: toplayıcılar.

Yarım Toplayıcı (Half Adder)

İşlev: İki biti (A ve B) toplar, elde bitini dikkate almaz.

Çıkışlar:

- **Sum (Toplam):** A ⊕ B (XOR)
- **Carry (Elde):** A · B (AND)

A	B	Sum	Carry
0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	1

Sadece en sağdaki (en düşük anlamlı) bit için kullanılabilir.

Tam Toplayıcı (Full Adder)

İşlev: Üç biti toplar: A, B ve önceki adımdan gelen Carry-in (Cin).

Çıkışlar:

- **Sum:** A ⊕ B ⊕ Cin
- **Carry-out (Cout):** (A·B) + (Cin·(A⊕B))

A	B	Cin	Sum	Cout
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

Tam toplayıcı, çok-bitli toplama işlemlerinin temel yapı taşıdır.

Ripple-Carry Adder: 32-bit Toplama

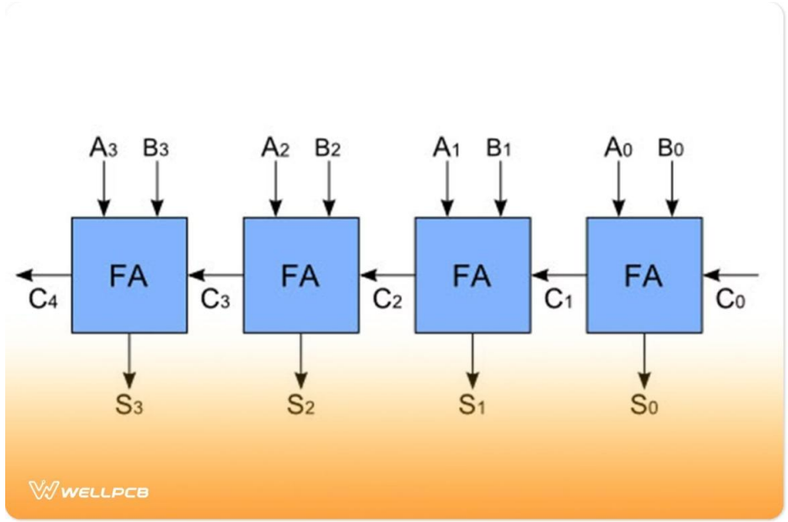
Tek bir tam toplayıcı yalnızca 1 bit toplar. Peki 32-bit'lik iki sayıyı nasıl toplarız?

Zincirleme Bağlama

Ripple-carry adder, tam toplayıcıları zincirleme bağlayarak oluşturulur:

1. 32 tane tam toplayıcı yan yana dizilir
2. Her toplayıcının Carry-out çıkışı, bir sonraki toplayıcının Carry-in girişine bağlanır
3. En sağdaki (LSB) toplayıcının Cin = 0'dır
4. Elde biti (carry) soldan sağa "dalgalanır" (ripple)

Örnek: 4-bit toplama için 4 tam toplayıcı zincirlenir. Her bit pozisyonu için bir toplayıcı kullanılır.



Performans Sorunu

Yavaş Hesaplama

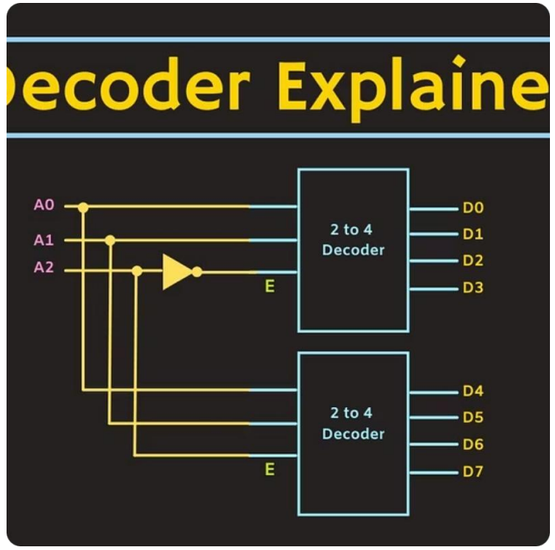
32. bitin sonucu **doğru olması için**, eldenin 1. bitten 32. bite kadar "dalgalanmasını" beklemek gerekir. Bu sıralı işlem, hesaplamayı yavaşlatır.

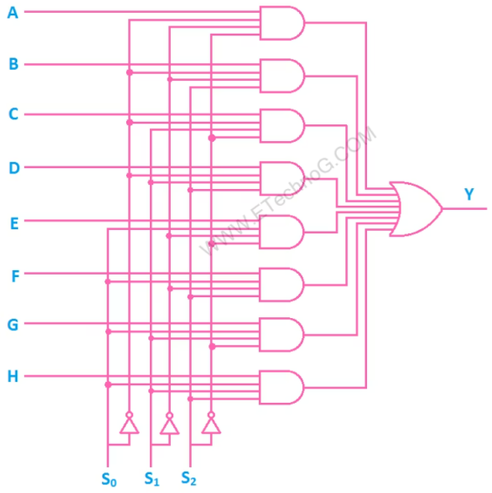
Modern Çözümler

Modern işlemciler **carry-lookahead adder** ve **carry-select adder** gibi daha hızlı toplayıcı mimarileri kullanır. Bunlar, elde bitini paralel olarak hesaplayarak gecikmeyi azaltır.

Decoder ve Multiplexer

Veri yönlendirme ve seçme işlemleri için kullanılan iki temel kombinasyonel blok.





8 to 1 Multiplexer

Decoder (Kod Çözücü)

İşlev: *n* girişi alır, 2n çıkıştan **sadece birini** aktif eder.

Örnek: 2'den 4'e decoder:

- 2 giriş biti (A, B)
- 4 çıkış hattı (Y0, Y1, Y2, Y3)
- Giriş değerine göre sadece ilgili çıkış 1 olur

Kullanım: Bellek adresi seçimi, yonga seçimi (chip select)

Multiplexer (MUX - Çoklayıcı)

İşlev: *n* bit "seçim" girişine göre, 2n veri girişinden **birini seçip** tek çıkışa yönlendirir.

Örnek: 4'ten 1'e MUX:

- 2 seçim biti (S0, S1)
- 4 veri girişi (D0, D1, D2, D3)
- 1 çıkış (seçilen veri)

Kullanım: Veri yolu yönlendirmesi, CPU'da register seçimi

Karşılaştırma

Özellik	Decoder	Multiplexer
Giriş	n adres biti	n seçim + 2n veri biti
Çıkış	2n hat (biri aktif)	1 hat (seçilen veri)
Amaç	Adres → Hat seçimi	Çok veri → Tek hat

Sıralı Devreler: Hafızanın Doğuşu

Şimdiye kadar incelediğimiz kombinasyonel devrelerin bir "hafızası" yoktu. Çıkış sadece o anki girişlere bağlıydı. Peki bilgisayar "durumu" (state) nasıl saklar?

Geri Besleme

Çıkışı girişe geri besleyerek (feedback) "hafıza" yaratılır. Devre, önceki durumunu "hatırlar".

Temel Bellek Elemanları

Latch ve **Flip-Flop**, en temel bellek elemanlarıdır. Her biri **1 bit** bilgi saklar.

Saat Senkronizasyonu

Flip-flop'lar, "saat darbesi" (clock) ile senkronize çalışır. Durum değişikliği sadece saat kenarında olur.

D-Tipi Flip-Flop

En yaygın kullanılan flip-flop türü:

- **D (Data) girişi:** Saklanacak bit değeri
- **Clock girişi:** Saat sinyali
- **Q çıkışı:** Saklanan değer

Çalışma: Saat darbesinin yükselen kenarında (0→1), D girişindeki değer Q'ya kopyalanır ve saklanır. Clock değişmediği sürece Q sabit kalır.

Register (Yazmaç)

Birden fazla flip-flop'un bir araya gelmesiyle **register** (yazmaç) oluşur:

- 8-bit register = 8 D flip-flop
- 32-bit register = 32 D flip-flop

CPU'nun içindeki registerlar, işlemci hızında veri saklayan en hızlı bellek birimleridir.

Durum Makineleri (FSM)

Asansör, trafik lambası, otomatik kapı veya bir işlemcinin kendisi gibi "durumları" olan sistemler nasıl tasarlanır?

Kırmızı

Araçlar dur. 30 saniye bekle.



Sarı

Hazırlan. 5 saniye bekle.

Yeşil

Geç. 30 saniye bekle.

FSM Bileşenleri

Durumlar (States)

Sistemin bulunabileceği **sonlu sayıda** farklı durum. Örnek: Kırmızı, Sarı, Yeşil.

Geçişler (Transitions)

Durumlar arası geçişler, **giriş koşullarına** bağlıdır. Örnek: 30 saniye geçtikten sonra Kırmızı → Sarı.

Çıkışlar (Outputs)

Her durumda üretilen sinyaller. Örnek: Kırmızı durumunda kırmızı ışığı yak.

Donanım Gerçekleme

Kombinasyonel Lojik (geçiş/çıkış mantığı) + **Flip-Flop'lar** (mevcut durumu saklamak için).

Modern işlemciler, aslında son derece karmaşık durum makineleridir. Her komut çalıştırılırken, işlemci belirli durumlardan (fetch, decode, execute, write-back) geçer.